

ENGINEERING DESIGN DOCUMENT · V1

SpecGuard

generate from spec · guard against drift

An AI-assisted API test tool that generates a review-ready test suite from an OpenAPI spec, then detects contract & behavior drift over time.

```
OpenAPI spec → [ Generate ] → review → pytest suite → CI
live responses → [ Baseline ] → [ Guard ] → drift report
```

Buildable spec — architecture, module contracts, data models, prompts, CLI, milestones
Stack: Python · pytest · jsonschema · LLM (Claude API or local via Ollama) · OpenAPI

Contents

1	Problem, goals & non-goals	p.3
2	Design principles	p.4
3	System architecture & data flow	p.5
4	Module breakdown & interfaces	p.6
5	Data models	p.9
6	The Generate half — how it works	p.10
7	The Guard half — how it works	p.12
8	CLI & CI integration	p.14
9	Project layout & build milestones	p.15
10	Risks, testing the tool, & extensions	p.16

1 Problem, goals & non-goals

WHAT SPECGUARD IS AND WHAT IT DELIBERATELY ISN'T

The problem. Writing API tests by hand is slow and uneven — engineers cover the happy path and skip the negative, boundary, and auth cases under time pressure. And once tests exist, the API drifts: a field goes nullable, an enum gains a value, a type changes. The tests still pass because they only assert what someone happened to think of, so **silent contract drift** reaches consumers before anyone notices.

SpecGuard attacks both halves with one coherent tool. The **Generate** half turns an OpenAPI spec into a thorough, review-ready pytest suite in minutes. The **Guard** half records real response shapes as a baseline and flags drift on every later run. One sentence: *generate the tests from the spec, then keep them honest as the API changes.*

Goals

- From an OpenAPI 3.x spec, generate runnable pytest modules covering happy path, validation failures, boundaries, and auth boundaries — as a *reviewed draft*, never blind-trust output.
- Record a per-endpoint response baseline (inferred JSON Schema + value stats) on demand.
- On later runs, diff live responses against the baseline and produce a structured drift report (added/removed/renamed fields, type changes, enum changes, value anomalies).
- Emit standard pytest + JUnit XML so it drops into any CI with no special glue.
- Work with either a hosted LLM (Claude API) or a local model (Ollama) behind one interface.

Non-goals (v1)

- No autonomous gating on AI output — a human confirms generated tests and promotes drift findings; the AI never silently fails a build.
- No load/performance testing (that's a separate tool).
- No GraphQL in v1 — REST/OpenAPI only; GraphQL is a documented extension.
- No UI. CLI-first; a dashboard is a later extension.
- Not a general fuzzer — generation is spec-driven, not random.

2 Design principles

THE RULES EVERY MODULE OBEYS

PRINCIPLE 1 -- AI IS NARROW AND REVIEWABLE, NEVER SILENT AUTHORITY

The LLM *suggests* tests and *proposes* drift findings. A human confirms before anything gates a build. An AI component that's confidently wrong and hides a real failure is worse than no tool, so every AI output is surfaced for review and there is always a deterministic path.

PRINCIPLE 2 -- DETERMINISTIC CORE, AI AT THE EDGES

The spec parser, schema inference, diff engine, and test runner are all deterministic and independently testable. The LLM is confined to two well-bounded jobs: turning a structured endpoint into test cases, and (optionally) explaining a drift in prose. Everything load-bearing is reproducible without the model.

PRINCIPLE 3 -- GENERATED CODE IS HUMAN-OWNED

Output is real pytest a person reads, edits, and commits — not a runtime black box. If the LLM is unavailable, the already-generated suite still runs. Generation is a one-time authoring aid, not a runtime dependency.

PRINCIPLE 4 -- PLUGGABLE MODEL, OFFLINE-CAPABLE

One `LLMProvider` interface with a Claude implementation and an Ollama implementation. Local models keep data in the environment and remove per-call cost — the same trade you'd make for any CI-embedded tool.

PRINCIPLE 5 -- STANDARD OUTPUTS, ZERO LOCK-IN

pytest + JUnit XML + JSON reports. SpecGuard produces artifacts every CI already understands; it never asks the pipeline to adopt a bespoke format.

3 System architecture & data flow

TWO PIPELINES SHARING A SPEC MODEL AND AN LLM PROVIDER

SpecGuard is two pipelines over shared infrastructure. Both consume the same parsed `EndpointModel`; both can use the same `LLMProvider`. They're independent — you can generate without ever guarding, or guard a hand-written suite without generating.

DATA FLOW (BOTH PIPELINES)

```
GENERATE pipeline
=====
openapi.yaml
-> SpecParser          -> List[EndpointModel]
-> CaseDesigner        -> List[TestCase]      (deterministic matrix +
                                   LLM-proposed extra cases)
-> TestRenderer        -> pytest .py files  (Jinja2 templates)
-> [ HUMAN REVIEW ]    -> committed suite

GUARD pipeline
=====
live API + suite
-> Runner              -> captured responses
-> SchemaInferer       -> inferred schema + value stats (baseline mode)
-> DriftEngine         -> diff(baseline, current)      (guard mode)
-> Reporter            -> drift_report.json + console + (opt) LLM summary
-> [ HUMAN CONFIRM ]   -> promote finding to a gating assertion
```

TWO MODES OF THE GUARD PIPELINE

Baseline mode (`specguard baseline`) records the current responses as the source of truth. **Guard mode** (`specguard guard`) compares live responses to that baseline and reports drift. Baseline is an explicit, human-triggered action — you never silently overwrite the contract you're guarding against.

4 Module breakdown & interfaces

EACH MODULE IS SMALL, DETERMINISTIC WHERE POSSIBLE, AND UNIT-TESTABLE

Module	Responsibility	Deterministic?
spec_parser	OpenAPI 3.x → EndpointModel list	Yes
case_designer	EndpointModel → TestCase list	Core yes, LLM adds extras
test_renderer	TestCase list → pytest source	Yes
llm	Provider interface + Claude/Ollama impls	No (isolated)
runner	Execute requests, capture responses	Yes
schema_inferer	Responses → inferred schema + stats	Yes
drift_engine	diff(baseline, current) → findings	Yes
reporter	Findings → JSON/console/JUnit	Yes
cli	Command surface, wiring	Yes

Note how little is non-deterministic: only the `llm` module, plus the optional extra-case suggestions in `case_designer`. Everything that gates or asserts is reproducible.

Key interfaces (write these first)

LLM/PROVIDER.PY -- THE SEAM THAT MAKES THE MODEL PLUGGABLE

```
from typing import Protocol

class LLMProvider(Protocol):
    def complete(self, system: str, user: str,
                 max_tokens: int = 2000) -> str: ...

class ClaudeProvider:
    def __init__(self, model="claude-sonnet-4-6"):
        import anthropic
        self.client = anthropic.Anthropic() # key from env
        self.model = model
    def complete(self, system, user, max_tokens=2000):
        resp = self.client.messages.create(
            model=self.model, max_tokens=max_tokens,
            system=system, messages=[{"role": "user", "content": user}])
        return "".join(b.text for b in resp.content if b.type == "text")

class OllamaProvider:
    def __init__(self, model="qwen2.5-coder"):
        self.model = model
    def complete(self, system, user, max_tokens=2000):
        import requests
        r = requests.post("http://localhost:11434/api/chat", json={
            "model": self.model, "stream": False,
            "messages": [{"role": "system", "content": system},
                         {"role": "user", "content": user}])
        return r.json()["message"]["content"]
```

SPEC_PARSER.PY -- DETERMINISTIC, NO LLM

```

from dataclasses import dataclass, field

@dataclass
class EndpointModel:
    method: str          # GET, POST, ...
    path: str             # /v1/orders/{id}
    operation_id: str
    path_params: list     # [{name, type, required}]
    query_params: list
    request_schema: dict | None # JSON Schema of the body
    response_schema: dict | None # JSON Schema of the 2xx body
    success_status: int     # 200 / 201 / 204
    requires_auth: bool

def parse_spec(path: str) -> list[EndpointModel]:
    import yaml
    spec = yaml.safe_load(open(path))
    endpoints = []
    for route, methods in spec["paths"].items():
        for method, op in methods.items():
            if method not in {"get", "post", "put", "patch", "delete"}:
                continue
            endpoints.append(_build_model(route, method, op, spec))
    return endpoints

```

CASE_DESIGNER.PY -- DETERMINISTIC MATRIX + OPTIONAL LLM EXTRAS

```

@dataclass
class TestCase:
    name: str
    kind: str          # happy | validation | boundary | auth | llm_extra
    method: str
    path: str
    body: dict | None
    headers: dict
    expected_status: int
    assertions: list    # structured, rendered into pytest asserts

def design_cases(ep: EndpointModel,
                 llm: LLMPProvider | None = None) -> list[TestCase]:
    cases = _deterministic_matrix(ep) # ALWAYS produced, no LLM needed
    if llm:
        cases += _llm_extra_cases(ep, llm) # e.g. domain edge cases from descriptions
    return cases

def _deterministic_matrix(ep):
    cases = [_happy(ep)]
    cases += _validation_cases(ep) # each required field missing / wrong type
    cases += _boundary_cases(ep)   # numeric/string min & max +/- 1
    if ep.requires_auth:
        cases.append(_auth_case(ep)) # no token -> 401
    return cases

```

WHY THE DETERMINISTIC FLOOR MATTERS

`_deterministic_matrix` guarantees a solid baseline suite even with the LLM turned off. The model only *adds* cases it infers from natural-language field descriptions (“must be a future date”, “cannot exceed the order total”). If the LLM hallucinates a bad case, review catches it; if it’s unavailable, you still get the floor.

5 Data models

THE THREE ARTIFACTS SPECGUARD READS AND WRITES

Three persisted artifacts. Keep them plain JSON so they diff well in Git and any tool can read them.

BASELINE.JSON -- THE RECORDED CONTRACT, PER ENDPOINT (GUARD)

```
{
  "version": 1,
  "recorded_at": "2026-07-22T00:00:00Z",
  "endpoints": {
    "GET /v1/orders/{id}": {
      "success_status": 200,
      "inferred_schema": {
        "type": "object",
        "required": ["id", "amount", "currency", "status"],
        "properties": {
          "id": {"type": "string"},
          "amount": {"type": "number"},
          "currency": {"type": "string"},
          "status": {"type": "string",
            "enum": ["created", "paid", "void"]}
        }
      },
      "value_stats": {
        "amount": {"min": 1.0, "max": 999999.0, "nullable": false},
        "status": {"observed_values": ["created", "paid", "void"]}
      }
    }
  }
}
```

DRIFT_REPORT.JSON -- THE OUTPUT OF GUARD MODE

```
{
  "generated_at": "2026-07-29T00:00:00Z",
  "findings": [
    {
      "endpoint": "GET /v1/orders/{id}",
      "severity": "breaking",
      "kind": "field_removed",
      "field": "currency",
      "detail": "was required in baseline, absent in current response"
    },
    {
      "endpoint": "GET /v1/orders/{id}",
      "severity": "warning",
      "kind": "enum_added",
      "field": "status",
      "detail": "new value 'refunded' not in baseline enum"
    },
    {
      "endpoint": "GET /v1/orders/{id}",
      "severity": "info",
      "kind": "field_added",
      "field": "updated_at",
      "detail": "new optional field; consider adding to schema"
    }
  ],
  "summary": {
    "breaking": 1,
    "warning": 1,
    "info": 1
  }
}
```

SEVERITY IS DETERMINISTIC, NOT AI-DECIDED

A removed or renamed required field is **breaking**. A new enum value or a widened type is a **warning**. A new optional field is **info**. The rules live in `drift_engine` as plain code, so the same drift always gets the same severity — the LLM only writes an optional human-readable explanation on top.

6 The Generate half — how it works

SPEC IN, REVIEWED PYTEST OUT

1. **Parse.** `spec_parser` turns the OpenAPI file into `EndpointModel` objects. Pure, testable, no LLM.
2. **Design cases.** `case_designer` builds the deterministic matrix, then optionally asks the LLM for extra cases it can infer from field *descriptions* the schema can't encode.
3. **Render.** `test_renderer` fills a Jinja2 template per endpoint, producing readable pytest that uses a client fixture — the client layer pattern, so tests read as intent.
4. **Review.** Output lands in `generated/` flagged for review. The human edits, deletes, or approves, then commits into the real suite.

THE LLM PROMPT FOR EXTRA CASES (SYSTEM + USER)

```
SYSTEM = """You are a senior test engineer. Given one API endpoint,
propose ADDITIONAL test cases that a JSON Schema cannot express but a
careful engineer would test, based on the natural-language field
descriptions. Return ONLY a JSON array of cases, no prose. Each case:
{"name","kind","body","expected_status","reason"}.
Do NOT restate cases already covered by required/type/enum validation."""

USER = f"""Endpoint: {ep.method} {ep.path}
Description: {ep.description}
Request schema: {json.dumps(ep.request_schema)}
Field notes: {ep.field_descriptions}

Return the JSON array of extra cases."""
```

RENDERED OUTPUT -- WHAT A HUMAN REVIEWS

```
# generated/test_orders.py (REVIEW BEFORE COMMITTING)
import pytest
from jsonschema import validate
from schemas import ORDER_SCHEMA

@pytest.mark.smoke
def test_get_order_happy_path(api):
    r = api.get("/v1/orders/o_1")
    assert r.status_code == 200
    validate(instance=r.json(), schema=ORDER_SCHEMA)

@pytest.mark.parametrize("field, value, expected", [
    ("amount", None, 422), ("amount", "x", 422),
    ("currency", "US", 422),
])
def test_create_order_validation(api, field, value, expected):
    payload = {"amount": 42.0, "currency": "USD"}
    payload[field] = value if value is not None else payload.pop(field, None)
    assert api.post("/v1/orders", json=payload).status_code == expected

# --- LLM-proposed (reason: description says exp must be a future date) ---
def test_expiry_must_be_future(api):      # REVIEW: confirm this is real
    r = api.post("/v1/cards", json={"exp_year": 2000})
    assert r.status_code == 422
```

THE REVIEW GATE IS A FEATURE, NOT FRICTION

Generated tests land in a separate `generated/` dir and are never auto-committed. The last LLM-proposed test above carries a `REVIEW` comment and its reason, so a human can confirm or delete it in seconds. This is what keeps the tool trustworthy.

7 The Guard half — how it works

BASELINE ONCE, THEN DIFF EVERY RUN

The Guard half is fully deterministic — the LLM is optional and only writes explanations. This is deliberate: the thing that decides whether your API drifted must be reproducible.

Schema inference (baseline mode)

Given N real responses for an endpoint, infer the schema by unioning observed structures: a field present in all responses is *required*; its type is the union of observed types; a string field with a small, stable value set becomes an *enum*; numeric ranges and nullability are recorded as `value_stats`.

SCHEMA_INFERER.PY -- THE CORE INFERENCE

```
def infer_schema(responses: list[dict]) -> dict:
    keys_in_all = set(responses[0])
    props = {}
    for r in responses:
        keys_in_all &= set(r)           # required = present everywhere
        for k, v in r.items():
            props.setdefault(k, set()).add(_json_type(v))
    schema = {"type": "object",
              "required": sorted(keys_in_all),
              "properties": {}}
    for k, types in props.items():
        prop = {"type": types.pop() if len(types) == 1 else sorted(types)}
        vals = [r[k] for r in responses if k in r]
        if _looks_like_enum(vals):      # few, stable string values
            prop["enum"] = sorted(set(vals))
        schema["properties"][k] = prop
    return schema
```

DRIFT_ENGINE.PY -- DETERMINISTIC DIFF WITH FIXED SEVERITIES

```
BREAKING = "breaking"; WARNING = "warning"; INFO = "info"

def diff(baseline: dict, current: dict) -> list[dict]:
    findings = []
    base_req = set(baseline["required"])
    cur_props = set(current["properties"])
    # a required field disappearing is breaking
    for field in base_req - cur_props:
        findings.append({"kind": "field_removed", "field": field,
                        "severity": BREAKING})
    # a new optional field is info
    for field in cur_props - set(baseline["properties"]):
        findings.append({"kind": "field_added", "field": field,
                        "severity": INFO})
    # type changes and enum changes
    for field in set(baseline["properties"]) & cur_props:
        b, c = baseline["properties"][field], current["properties"][field]
        if b.get("type") != c.get("type"):
            findings.append({"kind": "type_changed", "field": field,
                            "severity": BREAKING})
        if "enum" in b and "enum" in c and set(c["enum"]) - set(b["enum"]):
            findings.append({"kind": "enum_added", "field": field,
                            "severity": WARNING})
    return findings
```

THE DEMO THAT SELLS THE TOOL

Point SpecGuard at an API, run `baseline` , then have the API rename a required field and run `guard` — it reports a **breaking** drift at the exact field. That's the same ‘catch the break at the source’ story as contract testing, but with zero hand-written schemas.

8 CLI & CI integration

FOUR COMMANDS; DROPS INTO ANY PIPELINE

CLI SURFACE (BUILT WITH CLICK OR TYPER)

```
# generate a reviewable suite from a spec
specguard generate openapi.yaml --out generated/ --provider claude

# record the current responses as the contract baseline
specguard baseline --base-url https://api.staging.example.com \
  --out baseline.json

# compare live responses to the baseline; nonzero exit on breaking drift
specguard guard --base-url https://api.staging.example.com \
  --baseline baseline.json --report drift_report.json \
  --fail-on breaking

# run the generated suite and emit JUnit
specguard run generated/ --junitxml results.xml
```

CI WIRING (GITHUB ACTIONS)

```
- name: Guard against API drift
  run: |
    specguard guard \
      --base-url ${ env.STAGING_URL } \
      --baseline baseline.json \
      --report drift.json \
      --fail-on breaking          # gate ONLY on breaking; warn/info pass
- name: Publish drift report
  if: always()
  uses: actions/upload-artifact@v4
  with: { name: drift-report, path: drift.json }
```

--FAIL-ON IS HOW THE HUMAN KEEPS CONTROL

The pipeline gates only on the severity you choose. `--fail-on breaking` blocks a removed/renamed required field but lets a new enum value through as a warning. The tool never decides on its own to fail your build on an ambiguous change — you set the threshold.

9 Project layout & build milestones

WHAT TO BUILD, IN WHAT ORDER

REPOSITORY LAYOUT

```
specguard/  
  specguard/  
    __init__.py  
    spec_parser.py      # OpenAPI -> EndpointModel      (M1)  
    models.py           # EndpointModel, TestCase, Finding (M1)  
    llm/  
      provider.py       # LLMProvider protocol      (M2)  
      claude.py         # Claude impl              (M2)  
      ollama.py         # local impl               (M2)  
    case_designer.py    # matrix + LLM extras      (M2)  
    test_renderer.py    # Jinja2 -> pytest          (M2)  
    runner.py           # execute + capture        (M3)  
    schema_inferer.py   # responses -> schema        (M3)  
    drift_engine.py     # diff + severities        (M3)  
    reporter.py         # json / console / junit    (M3)  
    cli.py              # command surface          (M4)  
  templates/  
    test_module.py.j2  
  tests/                # SpecGuard's OWN tests  
  examples/  
    petstore.yaml       # a spec to demo against  
  pyproject.toml  
  README.md
```

Milestones

#	Milestone	Deliverable — done when...
M1	Spec parsing	<code>parse_spec()</code> turns Petstore into EndpointModels; fully unit-tested. No LLM yet.
M2	Generate half	<code>specguard generate</code> emits a runnable pytest suite from a spec, with the deterministic matrix working even with the LLM off.
M3	Guard half	<code>baseline</code> + <code>guard</code> detect a renamed field as breaking drift on a demo API.
M4	CLI + CI + polish	All four commands wired; JUnit output; README with the rename-a-field demo; published to PyPI.

SHIP M1-M2 FIRST, DEMO EARLY

The Generate half alone is a usable, demoable tool. Build it end to end before starting Guard, so you always have something that runs. This is the same ‘prove one slice before expanding’ approach that made your framework adoption work.

10 Risks, testing the tool & extensions

WHERE IT CAN GO WRONG, AND HOW YOU'D KNOW

Risks & mitigations

Risk	Mitigation
LLM invents a wrong test case	Deterministic floor stands alone; LLM extras land in review dir with a stated reason; never auto-committed.
Inferred schema is too strict (false drift)	Baseline from $N \geq 20$ responses; treat single-observation fields as optional; <code>--fail-on</code> defaults conservative.
Enum inference misfires on free-text fields	Only infer enum when the observed value set is small and stable relative to sample size; otherwise leave as plain string.
LLM cost / latency in CI	Generation is a one-time authoring step, not per-CI-run; Guard is LLM-free. Ollama option removes cost entirely.
Spec doesn't match reality	Guard compares against <i>real responses</i> , not the spec, so it catches spec-vs-implementation divergence too — a bonus.

Testing SpecGuard itself

- Unit-test the deterministic modules against a fixed example spec (Petstore) with golden outputs.
- Snapshot-test the renderer: same TestCase list must produce identical pytest.
- For the LLM module, test the *parsing* of model output (does it handle malformed JSON, extra prose?) with recorded fixtures, so tests don't call a live model.
- End-to-end: a tiny fake API + a spec + a deliberate field rename must produce exactly one breaking finding.

Extensions (post-v1)

- **Value-anomaly detection** — flag not just structure but suspicious values (a price that jumped 100x), using the recorded `value_stats`.
- **Failure-triage classifier** — the sibling tool from your list; reads a failed test and labels defect/flake/env. Shares the LLM provider.
- **GraphQL support** — infer from introspection instead of OpenAPI.
- **Web dashboard** — drift history over time as a trend view.
- **MCP server** — expose ‘generate tests for this endpoint’ as a tool an agent can call, reusing your self-healer MCP experience.

THE ONE-LINE PITCH FOR THE README

SpecGuard generates a review-ready API test suite from your OpenAPI spec, then guards against contract drift on every run — with the AI kept to a narrow, reviewable role and a fully deterministic core.